Project Update: C++ Core Sampler Implementation

Priyanshu Tiwari

May 23, 2025

Contents

1. Executive Summary 1
2. Key Accomplishments 1
3. Debugging and Problem-Solving 2
4. Validation and Performance 2
5. Next Steps 3

1. Executive Summary

This report outlines the successful completion of the foundational phase of the C++ core sampler reimplementation for the `dirichletprocess` package. The primary goal of this phase was to establish a robust C++ backend and ensure seamless integration with the existing R code, as detailed in the project proposal.

The core C++ infrastructure is now fully operational, validated, and ready for the implementation of the main MCMC algorithms. Key build issues have been resolved, and initial C++ components are demonstrating significant performance improvements over their R counterparts.

---

2. Key Accomplishments

2.1. C++ Backend and R/C++ Interface

A solid foundation for the C++ implementation has been established.

- **C++ Class Structure:** Header files for core classes (`DirichletProcessBase`, `MixingDistribution`, `NormalDistribution`) have been created, mirroring the R S3 structure as planned.
- **Robust Build System:** The R package's build system has been correctly configured (`NAMESPACE`, `DESCRIPTION`) to handle `Rcpp` and `RcppArmadillo` integration. This resolves earlier issues where C++ functions were not visible to R.
- **Functional R/C++ Interface:** R can now successfully call the newly implemented C++ functions via `Rcpp`. The `useDynLib` directive has been correctly added, ensuring all exported C++ functions are registered and available.

2.2. Initial Component Implementation

The first critical component of the MCMC sampler has been reimplemented in C++.

- **Likelihood Functions:** The `Likelihood` function for Normal and Multivariate Normal distributions has been implemented in C++.
- **Utility Framework:** The C++ backend includes a functional benchmarking and memory-tracking framework.

---

3. Debugging and Problem-Solving

The initial phase presented a significant challenge with the R package's build process. C++ functions were compiled but were not accessible from R, leading to `.Call()` errors.

- **Problem:** The root cause was identified as a missing `useDynLib(dirichletprocess, .registration = TRUE)` directive in the package's `NAMESPACE` file.
- **Solution:** The issue was resolved by adding the appropriate `#' @useDynLib` roxygen2 comment to the R source code and regenerating the documentation with `devtools::document()`. This corrected the `NAMESPACE` file, allowing R to properly register and find the C++ functions at load time.

This debugging process was crucial for creating the stable build environment we have now.

---

4. Validation and Performance

A comprehensive test suite was created to validate the C++ foundation. The suite confirms correctness by comparing C++ output to the original R functions and measures performance gains.

All foundational tests now pass with 100% success.

```
 Basic Infrastructure         PASSED
 Normal Likelihood            PASSED
 Class Structure              PASSED
 R/C++ Integration            PASSED
 Tools & Utilities            PASSED

Overall: 5/5 tests passed (100.0%)
```

The initial performance benchmarks are very promising. The C++ implementation of the normal likelihood function is over **1.6x faster** than the equivalent vectorized R `dnorm` function.

Table 1: Performance Comparison: Normal Likelihood

Implementation	Median Time (µs)
R (dnorm)	0.56
C++ (normal_likelihood_cpp)	0.34

This confirms that the C++ backend is not only correct but also provides the expected performance boost.

---

5. Next Steps

With the foundation validated, the project is perfectly positioned to proceed with the core MCMC logic. The immediate plan is:

1. **Complete the Normal Distribution Model:** Implement the `PriorDraw` and `PosteriorDraw` functions in C++.
2. **Implement Core Sampler Steps:** Implement the C++ versions of `ClusterComponentUpdate` and `ClusterParameterUpdate` for the conjugate Normal model.
3. **End-to-End Test:** Perform a full MCMC iteration using only C++ components for the Normal distribution to validate the entire architecture.

Once the Normal distribution is fully accelerated, this pattern will be replicated for the other distributions as outlined in the project timeline.